

AI Assisted Coding Assignment- 13.5

2303A51886 : 30 SAI SPURTHI. B 27.02.2026

Task Description 1(Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.
- o Refactor the code to pass values using function parameters.
- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

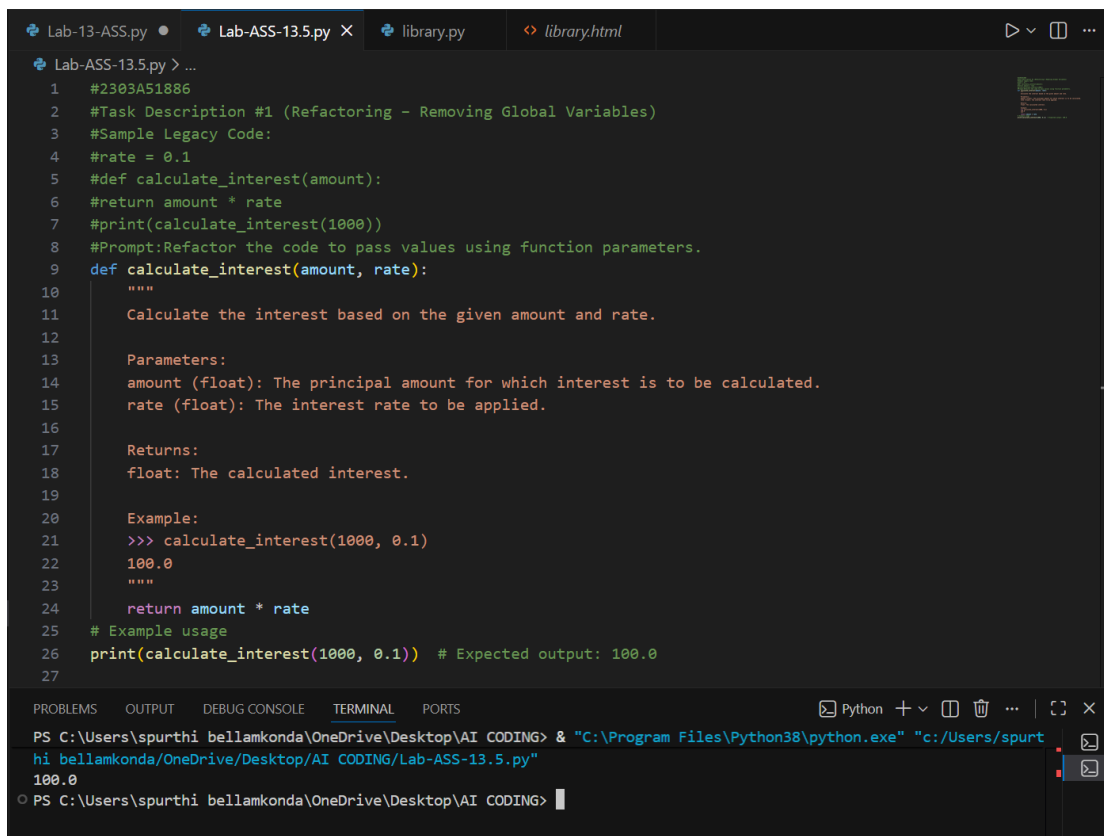
```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a configuration structure.



```
Lab-13-ASS.py • Lab-ASS-13.5.py X library.py library.html
Lab-ASS-13.5.py > ...
1  #2303A51886
2  #Task Description #1 (Refactoring - Removing Global Variables)
3  #Sample Legacy Code:
4  #rate = 0.1
5  #def calculate_interest(amount):
6  #return amount * rate
7  #print(calculate_interest(1000))
8  #Prompt:Refactor the code to pass values using function parameters.
9  def calculate_interest(amount, rate):
10     """
11     Calculate the interest based on the given amount and rate.
12
13     Parameters:
14     amount (float): The principal amount for which interest is to be calculated.
15     rate (float): The interest rate to be applied.
16
17     Returns:
18     float: The calculated interest.
19
20     Example:
21     >>> calculate_interest(1000, 0.1)
22     100.0
23     """
24     return amount * rate
25 # Example usage
26 print(calculate_interest(1000, 0.1)) # Expected output: 100.0
27
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-13.5.py"
100.0
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

Task Description 2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if-elif-else logic into a cleaner structure.

- Focus Areas:

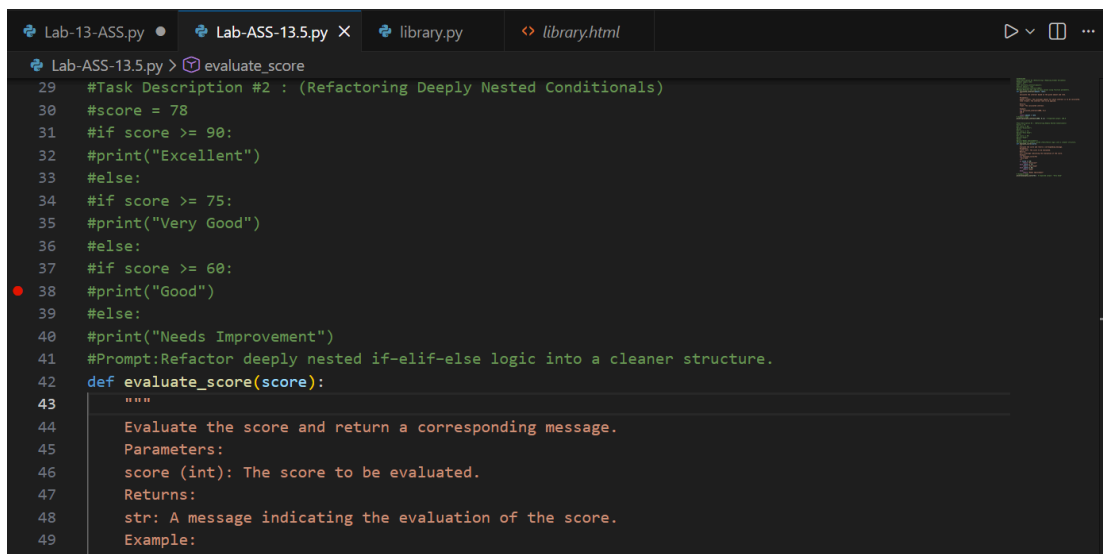
- o Readability
- o Logical simplification
- o Maintainability

Legacy Code:

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Expected Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.



The screenshot shows a code editor with a dark theme. The top bar has tabs for 'Lab-13-ASS.py', 'Lab-ASS-13.5.py', 'library.py', and 'library.html'. The main editor window shows the file 'Lab-ASS-13.5.py' with a cursor at line 42. The code is as follows:

```
29 #Task Description #2 : (Refactoring Deeply Nested Conditionals)
30 #score = 78
31 #if score >= 90:
32 #print("Excellent")
33 #else:
34 #if score >= 75:
35 #print("Very Good")
36 #else:
37 #if score >= 60:
38 #print("Good")
39 #else:
40 #print("Needs Improvement")
41 #Prompt:Refactor deeply nested if-elif-else logic into a cleaner structure.
42 def evaluate_score(score):
43     """
44     Evaluate the score and return a corresponding message.
45     Parameters:
46     score (int): The score to be evaluated.
47     Returns:
48     str: A message indicating the evaluation of the score.
49     Example:
```

The screenshot shows a VS Code editor with a Python file named 'Lab-ASS-13.5.py'. The code defines a function 'evaluate_score(score)' that returns a string based on the score. The function has a docstring and an example usage. The terminal output shows the function being called with the score 78, resulting in the output 'Very Good'.

```
42 def evaluate_score(score):
48     """
49     str: A message indicating the evaluation of the score.
50     Example:
51     >>> evaluate_score(78)
52     'Very Good'
53     """
54     if score >= 90:
55         return "Excellent"
56     elif score >= 75:
57         return "Very Good"
58     elif score >= 60:
59         return "Good"
60     else:
61         return "Needs Improvement"
62 # Example usage
63 print(evaluate_score(78)) # Expected output: "Very Good"
```

Terminal Output:

```
hi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-13.5.py
100.0
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/spurthi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-13.5.py"
100.0
Very Good
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle
- o Context managers
- o Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.


```
Lab-13-ASS.py • Lab-ASS-13.5.py X data1.txt data2.txt library.py library.html
Lab-ASS-13.5.py add_item
146 #Task 7 (Removing Hidden Side Effects)
147 #data = []
148 #def add_item(x):
149 #data.append(x)
150 #add_item(10)
151 #add_item(20)
152 #print(data)
153 #Prompt:Refactor function returning values instead of mutating globals.
154 def add_item(x, data_list):
155     data_list.append(x)
156     return data_list
157 data = []
158 data = add_item(10, data)
159 data = add_item(20, data)
160 print(data)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - []

PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/hi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-13.5.py"
Hello from file 2
Enter username: guest
Access Granted
40000.0
249999500000
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING> & "C:\Program Files\Python38\python.exe" "c:/Users/hi bellamkonda/OneDrive/Desktop/AI CODING/Lab-ASS-13.5.py"
100.0
Very Good
Hello from file 1
Hello from file 2
Enter username: guest
Access Granted
40000.0
249999500000
[10, 20]
```

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.

- Focus Areas:

- o Readability

- o Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
if any(c.isdigit() for c in password):
```

```
if any(c.isupper() for c in password):
```

```
print("Valid Password")
```

```
else:
```

```
print("Must contain uppercase")
```

```
else:
```

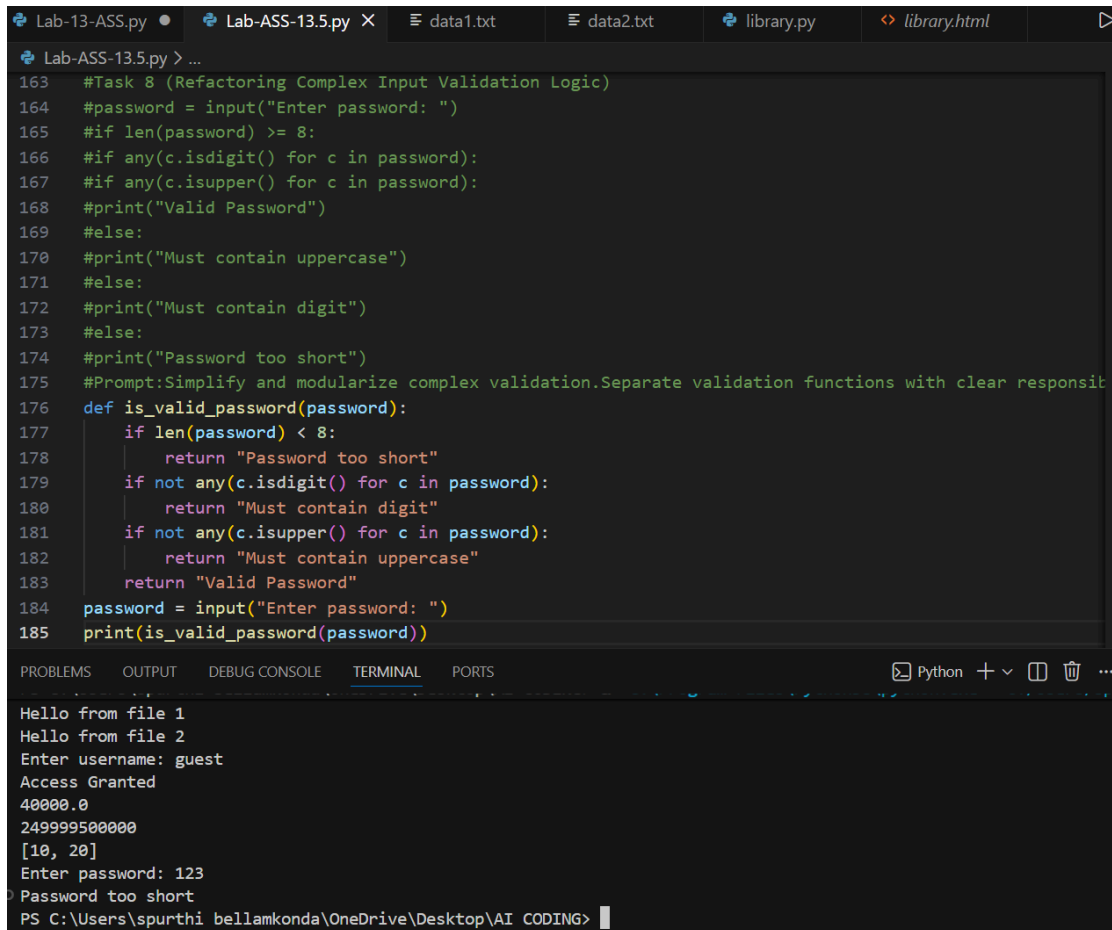
```
print("Must contain digit")
```

```
else:
```

```
print("Password too short")
```


Expected Outcome:

- o Separate validation functions with clear responsibility



The screenshot shows a VS Code editor with a Python file named 'Lab-ASS-13.5.py'. The code implements a password validation function. The terminal output shows the execution of the script, including prompts for username and password, and the resulting validation message.

```
Lab-13-ASS.py • Lab-ASS-13.5.py X data1.txt data2.txt library.py library.html
Lab-ASS-13.5.py > ...
163 #Task 8 (Refactoring Complex Input Validation Logic)
164 #password = input("Enter password: ")
165 #if len(password) >= 8:
166 #if any(c.isdigit() for c in password):
167 #if any(c.isupper() for c in password):
168 #print("Valid Password")
169 #else:
170 #print("Must contain uppercase")
171 #else:
172 #print("Must contain digit")
173 #else:
174 #print("Password too short")
175 #Prompt:Simplify and modularize complex validation.Separate validation functions with clear responsit
176 def is_valid_password(password):
177     if len(password) < 8:
178         return "Password too short"
179     if not any(c.isdigit() for c in password):
180         return "Must contain digit"
181     if not any(c.isupper() for c in password):
182         return "Must contain uppercase"
183     return "Valid Password"
184 password = input("Enter password: ")
185 print(is_valid_password(password))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ...
Hello from file 1
Hello from file 2
Enter username: guest
Access Granted
40000.0
249999500000
[10, 20]
Enter password: 123
Password too short
PS C:\Users\spurthi bellamkonda\OneDrive\Desktop\AI CODING>
```